Automated Image Intensity Balancer

MAJOR PROJECT REPORT

SUBMITTED IN PARTIAL FULFILMENT FOR DEGREE OF

BACHELOR OF ENGINEERING

In

(Computer Science and Engineering)

By

Mishra Nilesh Sanjaykumar and Jainil Mochi
pursued in

Department of Computer Science and Engineering
Faculty of Technology & Engineering

TO

सत्यं शिवं सुन्दरम्

THE MAHARAJA SAYAJIRAO UNIVERSITY OF BARODA
VADODARA, GUJARAT
APRIL, 2026

Automated Image Intensity Balancer

MAJOR PROJECT REPORT

SUBMITTED IN PARTIAL FULFILMENT FOR DEGREE OF

BACHELOR OF ENGINEERING

In

(Computer Science and Engineering)

By

Mishra Nilesh Sanjaykumar and Jainil Mochi
pursued in

Department of Computer Science and Engineering
Faculty of Technology & Engineering

TO

THE MAHARAJA SAYAJIRAO UNIVERSITY OF BARODA
VADODARA, GUJARAT

Guided By

Internal Guide
Prof. Meghana Desai ,
Department of CSE,
Faculty of technology and Engineering,
The MSU, Vadodara.

External Guide
Mr. Vijay Jariwala,
Software R & D Team Lead
STPL, Surat.

Department of Computer Science & Engineering Faculty of Technology & Engineering

The Maharaja Sayajirao University of Baroda Vadodara.

CERTIFICATE

This is to certify that **Mishra Nilesh SanjayKumar and Jainil Mochi** a student of Eighth Semester of BE- IV, **Department of Computer Science & Engineering, Faculty of Technology & Engineering, Baroda**, has successfully pursued a project titled **"Automated Image Intensity Balancer"** during the period from JANUARY 2026 to JUNE 2026, and has fulfilled the requirements towards the degree of Bachelor of Computer Science & Engineering.

Prof. Meghana Desai	**Dr. Apurva Shah**
Internal Project Guide	**Head of the Department**

Date: 18-04-2026

Place: Vadodara

ACKNOWLEDGEMENT

I am highly grateful to Prof. (Dr.) Dhanesh Patel, Dean, Faculty of Technology & Engineering for providing this opportunity to carry out the major project work at STPL, Surat.

The constant guidance and encouragement received from Prof. (Dr.) Apurva M. Shah, Head of CSE Department has been of great help in carrying out the project work and is acknowledged with reverential thanks.

I would like to express a deep sense of gratitude and thanks profusely to Prof. Meghana Desai, without her wise counsel and able guidance, it would have been impossible to complete the project in this manner.

I express gratitude to other faculty members of the computer science and engineering department of CSE for their intellectual support throughout the course of this work.

Finally, I am indebted to all whosoever have contributed in this report work.

Mishra Nilesh SanjayKumar

Jainil Mochi

COMPANY PROFILE

STPL

Incorporated in 1993, STPL has been at the forefront of developing cutting-edge technological solutions for the diamond industry. As a turnkey solution provider, we cater to every aspect of diamond manufacturing, from analysis and planning to processing, cutting, blocking, polishing, and safe trading. STPL is one of the few global companies offering comprehensive solutions, fulfilling the needs of clients across borders with unmatched precision and reliability.

As a trailblazer in the industry, STPL pioneered laser diamond cutting technology in India. Today, our advanced solutions leverage state-of-the-art laser technology, ensuring higher productivity at reduced costs. By optimising automation and eliminating resource-intensive finishing operations, we continue to set industry benchmarks.

Our product range includes diverse variants to suit various needs and budgets, making advanced technology accessible to all. STPL's commitment to engineering excellence, combined with our core values of quality, safety, integrity, and responsibility, has established us as a trusted global leader in the diamond processing industry.

Website

https://www.stpl.com/

Industries

3D Printing, Diamond Processing, Software

Company size

201-500 Employees

Headquarters

Surat, India

Founded

1993

ABSTRACT

The diamond processing industry requires extremely precise imaging systems to analyze diamonds during cutting, shaping, and polishing operations. Since diamonds have highly reflective surfaces and microscopic internal structures, even small variations in lighting or camera settings can significantly affect the captured images. Modern diamond processing machines therefore use multiple cameras placed around the diamond to observe it from different directions. However, maintaining consistent brightness and illumination across images captured from different cameras is a major challenge in such multi-camera setups.

In a typical inspection system, a diamond is placed at the center of the imaging setup and multiple cameras are arranged around it in a circular configuration. In the system studied during this project, three cameras are positioned at approximately **120-degree intervals**, ensuring that the diamond can be viewed from three different directions. Each camera is equipped with its own dedicated illumination source, usually in the form of a flashlight or LED light mounted near the camera. These lights help illuminate the diamond surface so that features such as inclusions, reflections, and edges can be clearly captured.

Although this arrangement allows comprehensive visualization of the diamond, it introduces a common industrial problem: **brightness inconsistency between camera feeds**. Differences in LED intensity, camera sensor characteristics, environmental reflections, or calibration drift can cause one camera to capture a brighter image while another camera captures a darker image of the same object. Such inconsistencies create difficulties when operators or automated systems attempt to compare images from multiple views. In diamond processing, where accurate visual interpretation is essential for determining cut quality, defects, and inclusion positions, inconsistent brightness can lead to incorrect analysis, production delays, or operator fatigue.

Manual adjustment of brightness levels for each camera feed is not a reliable solution. Industrial diamond processing machines often operate continuously in high-throughput environments, and manual calibration would require constant human intervention. Additionally, manual corrections are subjective and may introduce further inconsistencies. Therefore, there is a strong need for an **automatic image intensity balancing system** that can ensure consistent brightness across all camera images without human involvement.

To address this challenge, an **Image Intensity Balancing System** was designed and developed during a five-week internship at **Sahajanand Technologies Private Limited (STPL), Surat**, a leading company in diamond processing automation. The system is implemented as a standalone desktop application using **Delphi (Object Pascal) and the Visual Component Library (VCL)** framework. The software loads three bitmap images simultaneously, where one image is treated as the **reference image** and the other two images are automatically adjusted to match its brightness characteristics.

The system uses a **grid-based image processing approach** to analyze brightness differences across the images. Instead of analyzing the entire image as a single unit, each image is divided into smaller grid cells based on a user-defined grid size. For every grid cell, the system calculates the **average brightness value of the pixels** within that region. This local analysis allows the algorithm to detect brightness variations across different parts of the image rather than relying only on global brightness values.

To obtain accurate brightness measurements, the system applies the **ITU-R BT.601 luminance formula**, which represents human visual perception of brightness. In this formula, the red, green, and blue color channels contribute different weights to the final luminance value. The brightness of each pixel is calculated using the expression:

$$\text{Brightness} = 0.299 \times \text{Red} + 0.587 \times \text{Green} + 0.114 \times \text{Blue}$$

This weighting reflects the fact that the human eye is more sensitive to green light than red or

blue light. By applying this formula, the system calculates perceptually accurate brightness values for each grid cell.

After computing the brightness values for all grid cells in the three images, the system determines the **difference in brightness between the reference image and the other two images** for each corresponding cell. These differences represent the intensity errors that need to be corrected. The system then applies an iterative correction algorithm to gradually adjust the brightness of the target images.

Instead of applying the full correction at once, the algorithm uses a **gradual half-step adjustment approach**. In each iteration, only half of the calculated error is applied to the pixel intensities. This method prevents sudden brightness changes and allows the system to converge smoothly toward balanced brightness levels. The iterative process continues until the brightness difference between corresponding grid cells becomes smaller than a predefined threshold value.

During the development of the system, several technical challenges were encountered and addressed. Initially, pixel values were accessed using the graphical interface method Canvas.Pixels, which internally calls operating system functions for every pixel access. This method proved extremely slow when processing large images containing millions of pixels. To improve performance, the implementation was optimized by directly accessing the image memory using **ScanLine pointers and PRGBTriple structures**, allowing the program to read and modify pixel values directly in memory. This optimization significantly improved the processing speed of the algorithm.

Another issue involved handling image boundaries when grid cells extended beyond the dimensions of the image. This problem was solved by restricting cell boundaries using mathematical functions to ensure that all pixel accesses remained within valid limits. Additionally, brightness corrections could potentially cause pixel values to exceed the valid range of 0 to 255. To prevent color distortion, a clamping function was applied to ensure that pixel values always remain within the valid range.

The software interface was also designed to remain responsive during long processing operations. Since iterative image processing can take several seconds per iteration, the program periodically processes system messages to prevent the user interface from freezing. This allows users to pause or stop the balancing operation if necessary.

Beyond the software implementation, the project also explored **hardware integration possibilities**. In industrial systems, brightness balancing can also be achieved by controlling the intensity of the light sources associated with each camera. Therefore, a control architecture was designed in which the software can communicate with a microcontroller responsible for adjusting LED brightness levels.

In this setup, the computer running the image balancing software sends brightness correction values to a microcontroller through **serial communication**. The microcontroller converts these values into **PWM (Pulse Width Modulation) signals** that control the intensity of LED lights connected to each camera. By adjusting the light output, the system can automatically correct brightness differences in real time. This creates a **closed-loop control system**, where camera images are analyzed by the software and the lighting system is adjusted accordingly to maintain consistent illumination.

In addition to the image balancing system, another research task was carried out during the internship related to **software performance optimization and cross-language integration**. The company maintains a large library of functions written in **C++**, which were traditionally accessed from the Delphi software using **Dynamic Link Libraries (DLLs)**. Although DLLs allow modular software design, calling functions through DLLs introduces runtime overhead due to dynamic loading and symbol resolution.

To improve performance, research was conducted to determine whether these C++ functions could be compiled into **object files (.obj)** and directly linked with the Delphi application. The

C++ code was compiled into **COFF object files**, which were then converted into **OMF format** to ensure compatibility with the Delphi linker. Tools such as **Turbo Dump (TDUMP)** were used to analyze the object files and identify the exported function symbols. This process allowed the Delphi application to call C++ functions directly without relying on DLL-based runtime linking.

The research also involved studying compatibility between C++ and Delphi data types, calling conventions, and memory layouts to ensure correct communication between the two languages. The results demonstrated that static linking of object files can reduce function call overhead and improve performance in computation-intensive applications.

Overall, the internship project successfully demonstrates the application of image processing techniques, software optimization methods, and hardware integration concepts to solve a real industrial problem. The developed Image Intensity Balancing System ensures consistent brightness across multi-camera images, improving visual analysis and reliability in diamond processing environments. Additionally, the research on integrating C++ object files with Delphi provides a pathway for improving software performance in industrial automation systems. The project therefore contributes both practical engineering solutions and valuable research insights that can support future development of advanced imaging and automation systems in the diamond processing industry.

TABLE OF CONTENTS

chapter	Page No.
Certificate	***i***
Acknowledgement	***ii***
Company Profile	***iii***
Abstract	***iv***
Table of Contents	***v***
List of Figures	***vi***
Chapter 1: Introduction	**2**
1.1 Introduction to Project	2
1.2 Project Category	2
1.3 Objectives	3
1.4 Identification/Reorganization of Need	4
1.5 Unique Features of the System	4
1.6 Existing System	5
1.7 Proposed System	6
1.8 Unique Features of the System	6
Chapter 2: Requirement Analysis and System Specification	**8**
2.1 Feasibility study (Technical, Economical, Operational)	8
2.2 Software Requirement Specification Document	9
2.3 Validation	10
2.4 Expected hurdles	12
Chapter 3: System Design	**13**
3.1 Design Approach (Function oriented or Object oriented)	13
3.2 Detail Design	13
3.3 System Design using various Structured analysis and design tools	14

Chapeter 4 :Research: Integration of C++ Functions with Delphi 20

Chapter 5: Implementation, Testing and Maintenance 23

5.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation 23

5.2 Coding standards of Language used 24

Chapter 6: Results and Discussions 26

6.1User Interface Representation 26

6.2Snapshots of Application and Database tables 28

Chapter 7: Workflow During an Internship 33

Chapter 8: Conclusion and Future Scope 35

Data Dictionary 37

References

LIST OF FIGURES

Fig. No.	Figure Description	Page No.
1	Data Flow Diagram	16
2	Flow Chart	17
3	Old Runtime DLL Approach vs New Static .OBJ Integration	18
4	Complete System Architecture of Camera Intensity Balancer	19
5	Whole prototype Image	20
6	Prototype Image	21

Chapter 1 : Introduction

1.1 Introduction to Project

Modern industrial automation systems rely heavily on machine vision technologies to perform precise inspection, alignment, and quality control tasks. In industries such as diamond processing, high-resolution imaging systems capture multiple views of an object using several cameras positioned at different angles. These visual inputs are used by operators and software systems to analyze inclusions, plan cutting operations, and monitor machining processes.

Sahajanand Technologies Private Limited (STPL), headquartered in Surat, Gujarat, is a global leader in laser-based diamond processing machinery. Their machines such as **LAZER AIM**, **LAZER LEAP**, **MagikCut**, and **RoboShine** integrate advanced laser systems, robotic actuators, and industrial camera setups to automate the diamond manufacturing process. These machines operate continuously in high-precision environments where accurate visual data is critical for achieving maximum yield and minimizing diamond loss.

However, during operation of such systems, **intensity inconsistencies between camera feeds** may occur due to variations in lighting conditions, camera calibration differences, sensor drift, or environmental factors. When multiple images are displayed together, these inconsistencies make visual comparison difficult and may affect the accuracy of machine-vision algorithms.

To address this issue, the **Image Intensity Balancing System** was developed as a prototype desktop application during the internship at STPL. The system automatically analyzes brightness differences between images and adjusts them to achieve balanced intensity levels. It works by dividing images into grid regions, computing perceptual luminance values, and iteratively applying corrections until the intensity difference falls below a defined threshold.

The prototype was implemented using **Delphi / Object Pascal**, allowing efficient pixel manipulation and fast execution suitable for real-time industrial applications. The system provides a user interface that allows operators to load multiple images, monitor grid-level intensity values, and control the balancing process through play, pause, and stop operations.

This project demonstrates how digital image processing techniques can be applied to improve the reliability of industrial machine vision systems and contribute to automation in precision manufacturing environments.

This project also involves technical research related to integrating existing C++ functions into a Delphi-based system. The research focuses on compiling C++ code into object files and linking them directly with the Delphi application in order to improve execution performance.

1.2 Project Category

The **Image Intensity Balancing System** falls under the category of **Application/System Development with Industrial Automation Support**.

It is a desktop-based software application designed to assist machine-vision systems used in industrial diamond processing machines. The project combines concepts from:

•Digital Image Processing
• Industrial Automation
• Desktop Software Development

Additionally, the system uses several computational functions written in C++ that are currently accessed through DLL or executable files. Calling these functions through external files introduces additional execution overhead and increases the time required for processing.
Therefore, there is a need for an improved system that can automatically balance image intensity levels while also optimizing the integration of existing computational functions to improve performance.

1.3 Objectives

The main objectives of this project are:

- To develop a desktop application capable of loading and processing multiple images simultaneously.
- To design a grid-based image analysis technique that divides images into smaller regions for localized brightness comparison.
- To compute perceptual luminance values using the weighted RGB formula $L = 0.299R + 0.587G + 0.114B$.
- To detect intensity differences between corresponding regions of reference and target images.
- To implement an iterative half-step correction algorithm to gradually balance image brightness.
- To optimize pixel processing using pointer-based access for faster computation.
- To design a responsive graphical user interface that allows users to control the balancing process
- To research and implement a method for integrating C++ functions with a Delphi application using object files.
- To optimize system performance by reducing execution overhead associated with DLL-based function.

1.4 Problem Formulation

In multi-camera industrial vision systems, images captured from different cameras may exhibit varying brightness or intensity levels. These variations can arise due to several factors, including:

• Uneven lighting conditions in the workspace
• Differences in camera sensor characteristics
• Calibration errors between cameras
• Environmental changes affecting illumination
• Aging of imaging hardware components

When such images are displayed simultaneously, the visual mismatch can make it difficult for operators to perform accurate comparisons. Additionally, automated image-processing algorithms may produce inconsistent results when the intensity levels vary significantly.

Therefore, the problem addressed in this project is the **automatic balancing of intensity levels between multiple images** so that their brightness values become consistent while preserving image details

The project primarily focuses on software-level improvements that assist in image processing and brightness normalization. It also includes technical research related to improving the integration of existing C++ functions within the Delphi-based system environment.

Although the system is designed for industrial applications, the implementation is limited to the development and testing of the intensity balancing mechanism and the integration of computational functions. The project does not include hardware modifications to the camera setup or lighting environment.

1.5 Identification / Reorganization of Need

During the internship at **Sahajanand Technologies Pvt. Ltd.**, it was observed that several machine-vision applications rely on multiple cameras to capture different views of diamonds during analysis and processing. These camera feeds are displayed on control panels where operators monitor the alignment and condition of the diamond.

In such setups, slight differences in brightness between displays can affect visual interpretation and may lead to incorrect judgments when comparing different views of the same object.

Manual brightness adjustment is not practical in high-throughput industrial environments where machines operate continuously and process large numbers of images. Therefore, an automated solution is required that can:

• Detect brightness differences automatically
• Apply safe and gradual intensity corrections
• Maintain visual consistency across multiple image feeds
• Operate efficiently without slowing down machine operation

The third stage focused on integrating existing C++ functions into the Delphi application. Initially, these functions were accessed through DLL files. However, research was conducted to

explore the possibility of compiling the C++ code into object files and linking them directly with the Delphi system.

The research process involved compiling the C++ source code into object files, converting the generated COFF format into OMF format compatible with Delphi, and analyzing symbol definitions using tools such as **Turbo Dump (TDUMP)**. Various compiler settings were investigated to ensure that the required functions were properly exported as public definitions.

Finally, the system was tested to verify that the intensity balancing mechanism and the integrated C++ functions operated correctly within the Delphi environment.

1.6 Existing System

In the current industrial machine-vision setup used in diamond processing machines, images are captured through multiple cameras for inspection and monitoring purposes. However, due to variations in camera sensors, lighting conditions, and environmental factors, the images captured by different cameras often exhibit differences in brightness and intensity. These inconsistencies can negatively affect the accuracy of image analysis and reduce the reliability of machine-vision based inspection systems.

At present, brightness differences between images are typically corrected through **manual adjustments**. Operators modify camera settings, lighting parameters, or display configurations in order to achieve acceptable brightness levels. This process requires human intervention and relies heavily on the operator's visual judgment. Since no automated algorithm is used to balance image intensity, the results may vary between different operators and may not always produce consistent outcomes.

In addition to manual brightness adjustments, the existing system also utilizes several computational functions written in **C++**, which are accessed through **DLL (Dynamic Link Library) or executable files**. While these functions perform necessary processing tasks, calling them through external modules introduces additional runtime overhead due to dynamic loading and linking. This increases execution time and reduces overall system efficiency, especially when such functions are invoked repeatedly during image processing operations.

Because of these limitations, the current system may experience **inconsistencies in image brightness, reduced processing efficiency, and dependence on manual adjustments**. These challenges highlight the need for an improved system that can automatically balance image intensity levels while also optimizing the integration of computational functions for better performance.

1.7 Proposed System

The proposed **Image Intensity Balancing System** is designed as an internal, standalone desktop application built entirely in Delphi / Object Pascal using the VCL framework, tailored specifically for STPL's multi-camera imaging and display environments. It is intended to serve machine operators, R&D engineers, and quality assurance technicians by offering an automated, perceptually accurate, and hardware-safe solution for synchronizing brightness levels across multiple image feeds.

Key functionalities include the ability to load three images simultaneously, divide each into a configurable grid of user-defined cell size, compute grid-level average luminance weighted RGB formula, and iteratively apply half-step error corrections until full convergence is achieved. The system provides live visual feedback through numeric grid-average overlays and offers a real-time Play / Pause / Stop control interface for operator safety. Unlike manual adjustment methods, this system produces consistent, repeatable, and perceptually accurate results without any dependency on operator judgment or experience.

In addition, the system is backed by research into direct C++ OBJ file linkage with Delphi, addressing a performance bottleneck in STPL's existing software infrastructure where C++ functions were accessed through DLL files at runtime. The proposed approach of static OBJ linkage eliminates runtime loading overhead, improves execution speed, and keeps all compiled code within the main application executable — fully in line with STPL's internal IT policies on data security and offline operation.

1.8 Unique Features of the System

- **Perceptual Luminance Formula:** Uses the ITU-R BT.601 industry-standard weighted RGB formula (0.299R + 0.587G + 0.114B) for brightness calculation, accurately matching human visual perception rather than simple arithmetic averaging.
- **PRGBTriple Pointer Optimization:** Pixel access is performed using direct typed pointer arithmetic on raw ScanLine memory, delivering near-C-language processing speed and reducing per-iteration time from over 18 seconds to approximately 1.4 seconds — a 13× improvement over GDI-based pixel access.
- **Configurable Grid-Based Local Balancing:** Images are divided into user-defined N×N pixel cells and balanced cell-by-cell, preserving spatial brightness information across the image rather than applying destructive global adjustments.
- **Gradual Half-Step Error Convergence:** Each iteration applies only half the remaining brightness error, producing exponential decay convergence (80 → 40 → 20 → 10 → 5 → 2 → 1 → stop) that is safe for physical LED panel and PWM hardware integration.
- **EnsureRange Pixel Clamping:** All RGB channel corrections are individually clamped to the valid 0–255 range using EnsureRange(), preventing Byte arithmetic overflow and eliminating color corruption artifacts throughout the correction process.
- **Real-Time Play / Pause / Stop State Machine:** FRunning and FPaused boolean flags with a dedicated WaitIfPaused() function give operators immediate ability to freeze or abort the balancing process at any point — a non-negotiable safety requirement for industrial hardware deployment.

- **Live Grid-Average Numeric Overlays:** Brightness averages are rendered directly onto each image panel's canvas after every iteration, providing continuous real-time visibility into the convergence progress across all grid cells of all three images simultaneously.
- **Offline Desktop Compatibility:** Fully self-contained with no external runtime dependencies or cloud connectivity requirements, operating entirely within STPL's internal network and adhering to the company's data security and IT governance policies.
- **C++ OBJ Direct Linkage Research:** Documented a complete pipeline from COFF-to-OMF conversion, TDUMP-based PUBDEF symbol verification, compiler configuration for correct segment placement, and C++ to Object Pascal data type compatibility mapping — providing STPL's R&D team a clear technical pathway to eliminate DLL call overhead across their broader software stack.
- **Hardware Integration Ready:** Designed from the ground up for extension to physical LED display panels via serial PWM command packets to an Arduino or STM32 microcontroller, with a planned closed-loop camera feedback pipeline for real-world display calibration.

Chapter 2 : Requirement Analysis & System Specification

2.1 Feasibility study

The feasibility study is a crucial phase in the development of the Image Intensity Balancing System. It evaluates whether the proposed solution is practical and viable across three main aspects: technical, economical, and operational. The study ensures that the system can be developed within the given constraints while delivering the intended value.

Technical Feasibility

The Image Intensity Balancing System is technically feasible with the current infrastructure and technologies available. The system is developed using Delphi / Object Pascal with the VCL framework in RAD Studio, which is a reliable, industry-accepted tool for high-performance desktop application development on Windows. The VCL framework provides direct access to low-level bitmap memory through TBitmap and ScanLine, which is essential for the per-pixel processing operations required by the balancing algorithm. These technologies are well-supported and fully capable of handling the intended functionalities, including grid-based luminance computation, iterative error correction, real-time UI feedback, and operator state control.

Moreover, the development team possesses the necessary skills and expertise in Object Pascal programming, image processing concepts, and Windows application development to implement the system effectively. The additional research into C++ OBJ file linkage — involving COFF-to-OMF conversion, TDUMP symbol inspection, and cross-language data type compatibility — was technically validated and confirmed feasible within the internship period. There are no significant technical limitations anticipated that could hinder the successful implementation of the system.

.

Economical Feasibility

From an economic standpoint, the cost of development is minimal. The project is developed as part of an academic internship curriculum at STPL, and there are no commercial licensing fees or external development costs involved. RAD Studio is available through institutional and organizational licensing at STPL, and no additional third-party libraries or paid tools are required for the core system. The COFF-to-OMF conversion tools and TDUMP are freely available as part of the Embarcadero / Borland toolchain.

The system aims to reduce manual effort in image calibration across STPL's multi-camera workstations, which currently depends entirely on operator judgment and repeated manual adjustment. In a real-world scenario, automating this process can reduce machine setup time, eliminate recalibration errors, and improve throughput consistency — offering a measurable return on investment for STPL's production operations.

Operational Feasibility

The Image Intensity Balancing System is operationally feasible and is designed with simplicity and usability in mind. The system provides an intuitive graphical interface where operators can load images, specify a grid size, and initiate the balancing process with minimal training. Play, Pause, and Stop controls give operators direct real-time oversight of the process, and live numeric overlays on each image panel ensure that convergence progress is clearly visible at all times.

The system requires no changes to existing hardware or network infrastructure and can be deployed directly on STPL's existing Windows workstations. It is intended to be operated by machine technicians, R&D engineers, and quality assurance staff, ensuring accessibility and ease of use for all stakeholders involved.

2.2 Software Requirement Specification

Data Requirement:

The application does not expect the operator to have any specialized technical knowledge beyond basic computer usage. The user is required only to provide three valid bitmap images of identical dimensions and a numeric grid cell size value. All processing is performed internally by the application on in-memory TBitmap objects. No external database or network connection is required. For the C++ OBJ integration research component, the user is expected to have access to the C++ source files provided by STPL, a compatible C++ compiler (such as C++ Builder), and the Embarcadero TDUMP utility for symbol inspection..

Functional Requirement:

The system's performance requirements necessitate pixel access speeds sufficient for processing medium-to-large resolution images within acceptable time per iteration for an interactive industrial application. The use of PRGBTriple typed pointer arithmetic on raw ScanLine memory is mandatory to meet this requirement, as GDI-based Canvas.Pixels access was benchmarked at over 18 seconds per iteration — entirely unsuitable for real-time use. With PRGBTriple pointer iteration, per-iteration processing time is reduced to approximately 1.4 seconds on medium-resolution images, which is acceptable for the iterative convergence workflow. It is also important that the application remains fully responsive during processing — the UI must not freeze, buttons must remain clickable, and overlays must update visibly after each iteration. This is achieved by calling Application.ProcessMessages() at every iteration of the main balancing loop.

Performance Requirement:

The system's dependability requirement mandates consistent and reliable operation across all valid input image sets. The convergence loop must always terminate — either by reaching the threshold condition (absolute difference ≤ 1 for all cells) or by explicit user stop — and must never enter an infinite loop or crash due to boundary conditions. Grid cell boundary overflow is handled by clamping end coordinates using Min(), and pixel value overflow is handled by EnsureRange() on all three RGB channels independently. These measures ensure the application remains in a stable and consistent state throughout processing regardless of image dimensions or grid size chosen.

Dependability Requirements:

The project's dependability requirement mandates a highly reliable system that operates consistently and securely, ensuring data integrity and minimizing the risk of failures or disruptions. All tables in the database must have an extra column named isDeleted and marked as true when the user wants to delete some data so that data is never actually deleted from the database.

Maintainability Requirement:

The system's maintainability requirement entails ensuring that the codebase is modular, readable, and straightforward to extend or modify. Each major operation — grid average

computation, image balancing, intensity change application, UI overlay rendering, and state control — is encapsulated in a separate named procedure or function with a clear single responsibility. Inline comments are provided throughout the code to explain non-obvious logic, particularly in the pixel pointer arithmetic and convergence loop sections. This modular structure ensures that future enhancements — such as supporting live video frame input, integrating serial hardware communication for LED panel control, incorporating the Python-based flashlight intensity control layer, or extending the system to process additional image panels — can be implemented without restructuring the existing codebase.

Security Requirement:

The system's security requirement demands reliable and safe operation that protects both the software and connected physical hardware from unintended behaviour or misuse. All user inputs including grid cell size are validated before processing begins, and pixel correction values are strictly clamped to the valid 0–255 range using EnsureRange() to prevent arithmetic overflow from corrupting image data in memory.

On the hardware side, a critical safety decision was made to never apply the full brightness error in a single step. Doing so could cause sudden large jumps in voltage and current supplied to the camera lighting hardware, potentially damaging LED driver circuits or PWM controllers. Instead, only half the remaining error is applied per iteration, producing a gradual exponential decay (80 → 40 → 20 → 10 → 5 → 2 → 1 → stop) that allows hardware to adjust safely across multiple steps.

To further protect against situations where the loop may take unexpectedly long to converge or risk entering an uncontrolled state, a Play / Pause / Stop state machine is implemented using FRunning and FPaused boolean flags. This gives the operator immediate ability to freeze or completely abort the entire correction process at any point — ensuring that if any unexpected behaviour is observed in the lights, cameras, or connected circuits, operations can be halted instantly before any hardware damage occurs.

Look and Feel Requirement:

The project's "Look and Feel" requirement emphasizes creating an intuitive and visually appealing user interface (UI) that enhances user experience and engagement. This includes designing a modern and user-friendly interface within Frappe and ERPNext that aligns with industry standards and best practices in UI/UX design. Key aspects involve intuitive navigation, clear and concise layout, visually appealing graphics and typography, and customizable themes to reflect clients' branding. Moreover, ensuring responsiveness across devices and accessibility for users with disabilities are essential considerations to meet this requirement effectively.

2.3 Validation

Validation is an important component of the **Image Intensity Balancer System** to ensure that image data is processed correctly and that the intensity balancing algorithm produces reliable results. Multiple validation checks are implemented at different stages of the system to maintain data consistency and prevent errors during image processing. These validations ensure that the images being processed meet the required conditions and that the algorithm operates within defined limits.

Key types of validations implemented in the system include:

1. Image Input Validation

The system verifies that valid image files are selected before processing begins. Only supported image formats such as **JPEG, PNG, or BMP** are allowed. If the user attempts to process an unsupported or corrupted image file, the system displays an error message and prevents further processing.

2. Image Dimension Validation

Before performing intensity balancing, the system checks whether the reference image and the target images have compatible dimensions. This validation ensures that the images can be divided into corresponding grid regions for accurate comparison and correction.

3. Grid Region Validation

The system divides images into smaller grid blocks for localized intensity comparison. Validation ensures that the grid size selected by the user is valid and does not exceed the image boundaries. This prevents errors during region-based processing.

4. Pixel Value Range Validation

All pixel values are validated to remain within the acceptable **RGB range (0–255)** during processing. This ensures that brightness adjustments do not produce invalid pixel values or cause image distortion.

5. Luminance Calculation Validation

The system validates the calculation of brightness using the weighted RGB formula:

$$L = 0.299R + 0.587G + 0.114B$$

This ensures that luminance values are computed correctly for each grid region and used reliably during the balancing process.

6. Iteration Control Validation

During the balancing process, the system checks whether the brightness difference between corresponding regions has reached the defined threshold. If the difference becomes sufficiently small, the algorithm automatically stops further iterations to prevent unnecessary processing.

7. Processing Error Handling

Exception handling mechanisms are implemented to detect unexpected errors during image loading or processing. If any error occurs, the system prevents application crashes and provides appropriate feedback to the user.

8. Performance Validation

The system ensures that large images are processed efficiently by using **pointer-based pixel access and optimized loops**. This validation helps maintain stable performance when working with high-resolution images.

By implementing these validation mechanisms, the **Image Intensity Balancer System** ensures reliable image processing, maintains data integrity, and improves the accuracy of brightness balancing across multiple images.

2.3 Expected Hurdles

While developing the Image Intensity Balancing System, several technical challenges were anticipated and encountered during the development process.

The most significant initial hurdle was **pixel access performance**. Delphi's TBitmap provides multiple methods for reading and writing pixel data, but the most intuitive method — Canvas.Pixels[x, y] — internally invokes a GDI GetPixel system call for every individual pixel access. For a medium-resolution image processed with a 100-pixel grid size, this resulted in approximately 9 million system calls per iteration, producing per-iteration times exceeding 18 seconds — entirely unacceptable for an interactive application. Switching to PRGBTriple typed pointer arithmetic on raw ScanLine memory required a deeper understanding of Delphi's memory layout for bitmap data, pointer increment semantics, and careful boundary management, but ultimately reduced iteration time to approximately 1.4 seconds.

A second major hurdle was **correct luminance calculation**. The initial assumption that all three RGB channels contribute equally to perceived brightness led to visible inaccuracies in the balancing output. Researching and implementing the ITU-R BT.601 perceptual weighting standard required understanding the human visual system's non-uniform sensitivity to different wavelengths, and verifying that the weighted formula produced measurably better results than the simple arithmetic average on real image samples.

Cross-language integration between C++ and Delphi presented a significant research challenge. C++ object files compiled by modern compilers produce COFF format output, which is incompatible with Delphi's OMF-based linker. Converting between formats using coff2omf, then discovering that the required functions were appearing under COMDEF rather than PUBDEF in the TDUMP output, required detailed investigation into C++ Builder compiler segmentation settings before the issue could be resolved. Establishing correct data type and calling convention mappings between C++ and Object Pascal further required careful analysis to ensure parameters and return values were exchanged correctly without memory corruption.

UI responsiveness during processing was another challenge. Delphi's VCL runs on a single thread, meaning any tight loop running without yielding control to the Windows message pump causes the entire application to appear frozen. Identifying the correct placement of Application.ProcessMessages() calls within the nested loop structure — frequently enough to keep the UI responsive without introducing excessive overhead — required iterative testing.

Hardware integration introduced additional real-world challenges beyond software control. LED brightness does not scale linearly with PWM duty cycle due to gamma non-linearity, meaning a PWM value of 128 does not produce 50% of perceived brightness. Camera auto-exposure and white balance adjustments between frames created false brightness variations independent of the actual LED output. Determining the correct error constant for the Python-based light intensity control through hit-and-trial required repeated physical testing across a range of voltage and current values, as the relationship between software correction values and actual hardware output was non-trivial to model analytically. Ambient lighting in the testing environment further introduced noise into brightness readings, requiring the convergence threshold to be adjusted for stable real-world operation.

Chapter 3: System Design

3.1 Design Approach

The Image Intensity Balancing System follows an **Object-Oriented Design (OOD)** methodology, which is well suited for modeling the distinct components of the system and their interactions. OOD enables modularization of the system into self-contained units that encapsulate both data and behaviour, promoting code reusability, scalability, and ease of maintenance.

The primary entities of the system — the main application form, bitmap image objects, grid error arrays, pixel processing procedures, and the operator state machine — are modeled as logically separated components that interact in a well-defined sequence. The TForm1 class serves as the central controller, encapsulating all processing state including the three TBitmap objects (Bmp1, Bmp2, Bmp3), the two dynamic error arrays (ferror1, ferror2), the grid size parameter, the image dimensions (w, h), and the operational state flags (FRunning, FPaused). By applying encapsulation principles, all pixel-level operations, UI update routines, and hardware communication logic are kept within clearly bounded procedures, each with a single well-defined responsibility. This approach ensures that individual components can be modified, extended, or replaced independently without affecting the rest of the system.

3.2 Detailed Design

At the detailed design level, the system is composed of interrelated procedures and data structures, each handling a specific function within the processing pipeline.

The **TForm1** class serves as the top-level controller and encapsulates the complete application state. It holds references to the three TBitmap objects used for in-memory image processing, the two TError dynamic two-dimensional integer arrays that store per-cell brightness error values for the two target images, the integer variables for image width, height, and grid cell size, and the FRunning and FPaused boolean flags that govern the operator state machine.

The **getGridAvg()** function is responsible for computing the average luminance of a single grid cell within a given bitmap. It accepts the bitmap reference and the cell boundary coordinates as parameters, iterates over all pixels within the cell using PRGBTriple pointer arithmetic on raw ScanLine memory, applies the ITU-R BT.601 weighted luminance formula to each pixel, and returns the average value as an integer.

The **findGridAvg()** procedure iterates over the complete grid of all three images simultaneously, calling getGridAvg() for each corresponding cell triplet, computing the signed brightness difference between the reference image and each target image, and storing these error values into the ferror1 and ferror2 arrays for use in the subsequent correction step.

The **balanceImage()** procedure applies the brightness correction to a target bitmap. It iterates over all grid cells and calls changeIntensity() for each cell, passing half the stored error value for that cell — implementing the gradual half-step convergence scheme that ensures hardware-safe correction.

The **changeIntensity()** procedure performs the actual per-pixel brightness adjustment within a single grid cell. It walks the cell pixels using PRGBTriple pointer arithmetic and adds the correction value to all three RGB channels of each pixel, with EnsureRange() clamping applied independently to each channel to prevent byte overflow.

The **showAvg()** procedure handles live UI feedback by rendering the computed numeric average value for each grid cell directly onto the corresponding TImage panel's canvas after every iteration, giving the operator continuous real-time visibility into convergence progress.

The **WaitIfPaused()** function implements the operator state control logic. When FPaused is True, it enters a non-blocking spin loop calling Application.ProcessMessages() and Sleep(50) repeatedly until either FPaused is cleared by a Play button press or FRunning is set to False by a Stop button press, at which point it returns the current FRunning value to allow the main loop to exit cleanly.

The **Button2Click** procedure contains the main balancing loop — the heart of the algorithm. It validates inputs, initialises the error arrays, sets the state flags, and then repeatedly calls findGridAvg(), balanceImage(), and showAvg() in sequence, checking WaitIfPaused() at each iteration checkpoint, until the convergence condition is satisfied or the operator stops the process.

Error handling is managed throughout by try-except blocks that capture runtime exceptions and display user-friendly messages without crashing the application. This layered and modular structure promotes clarity and maintainability and provides a strong foundation for future extension — including live video frame input, serial PWM hardware communication, and integration with the Python-based flashlight intensity control and frame extraction pipeline.

3.3 System Design Using Structured Analysis and Design Tools

Data Flow Diagram

The system follows a linear data flow from operator input through image processing to hardware output. The operator provides three image files and a grid size value as inputs. These are loaded into TBitmap objects in memory. The grid average computation module reads pixel data from all three bitmaps and produces a set of per-cell brightness averages. The error calculation module compares the reference averages against the target averages and populates the ferror arrays with signed difference values. The balancing module reads the error arrays and applies half-step corrections back into the target bitmaps. The UI feedback module reads the updated bitmaps and renders numeric overlays onto the display panels. The convergence check module monitors whether all cell differences have fallen within the threshold and either triggers another iteration or terminates the loop. In the hardware integration phase, the corrected brightness values are additionally passed through the serial communication module as PWM command packets to the Arduino/STM32 microcontroller, which drives the physical LED panels. The Python subsystem runs in parallel, capturing live video frames, extracting individual images, and applying the calibrated error constant to control flashlight voltage and current per camera.

DFD — Automated Intensity Balancer

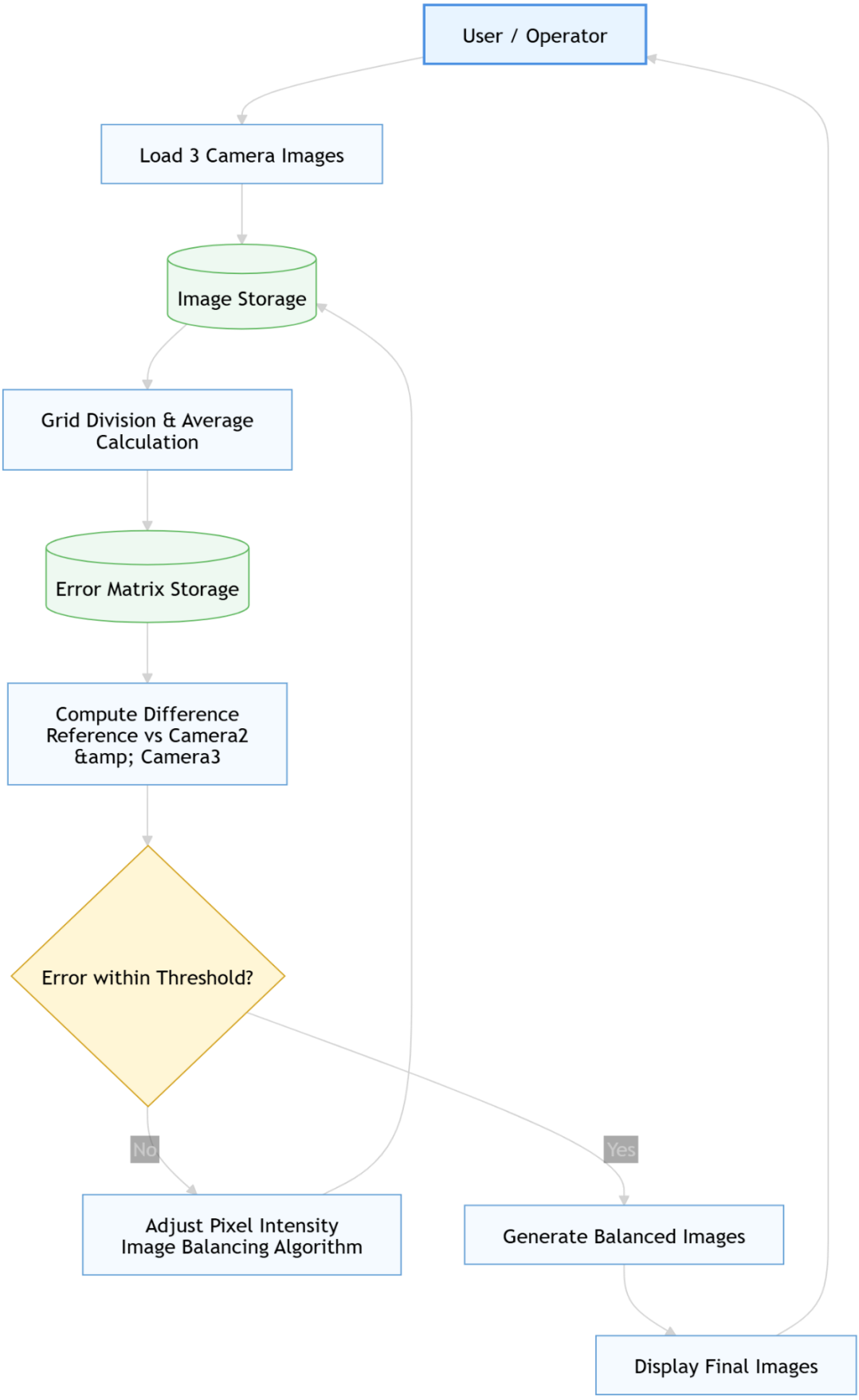


Figure 1. DFD diagram

Main balancing algorithm — flowchart

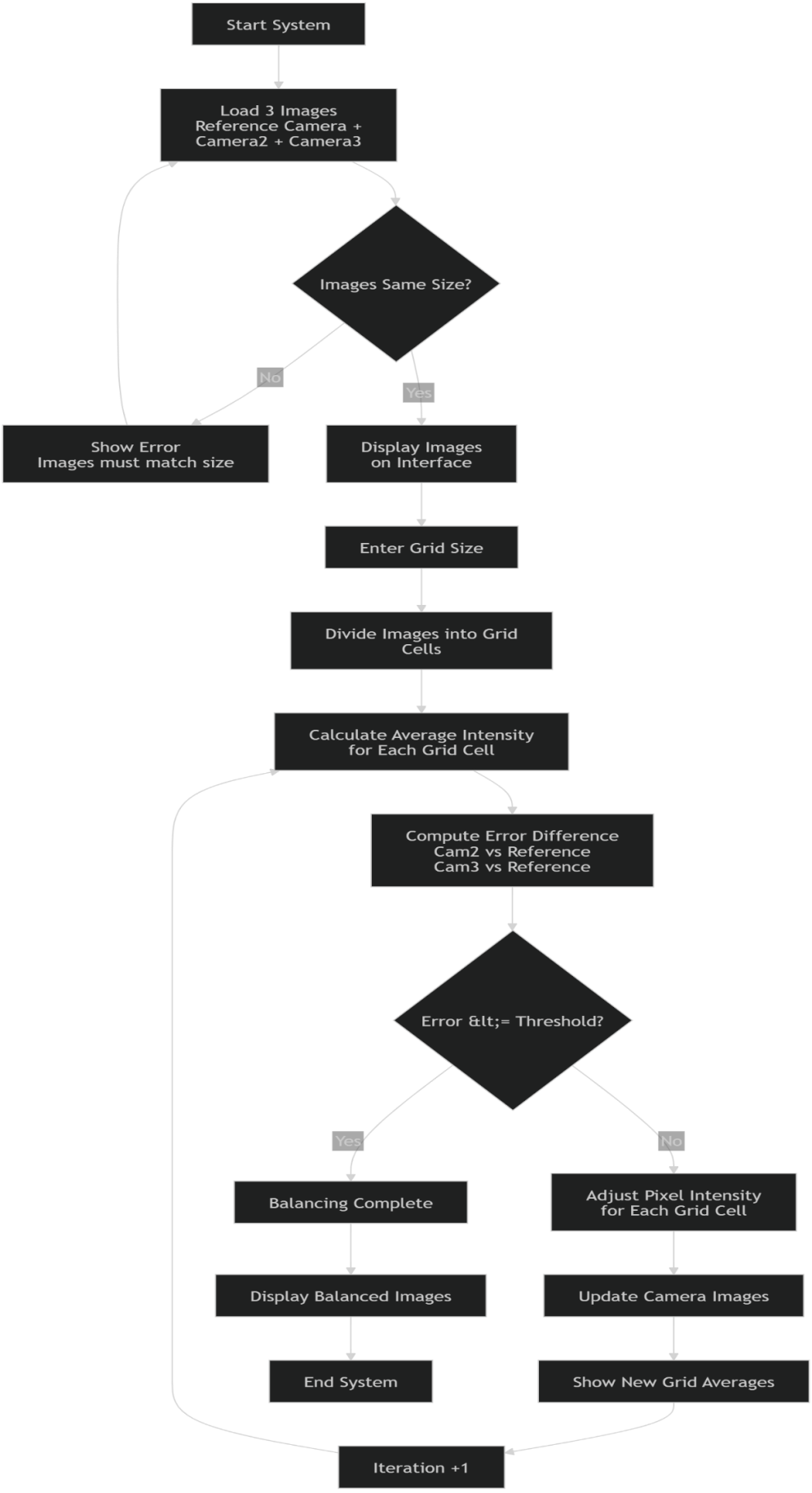


Figure 2. Flowchart diagram

Old Runtime DLL Approach vs New Static .OBJ Integration

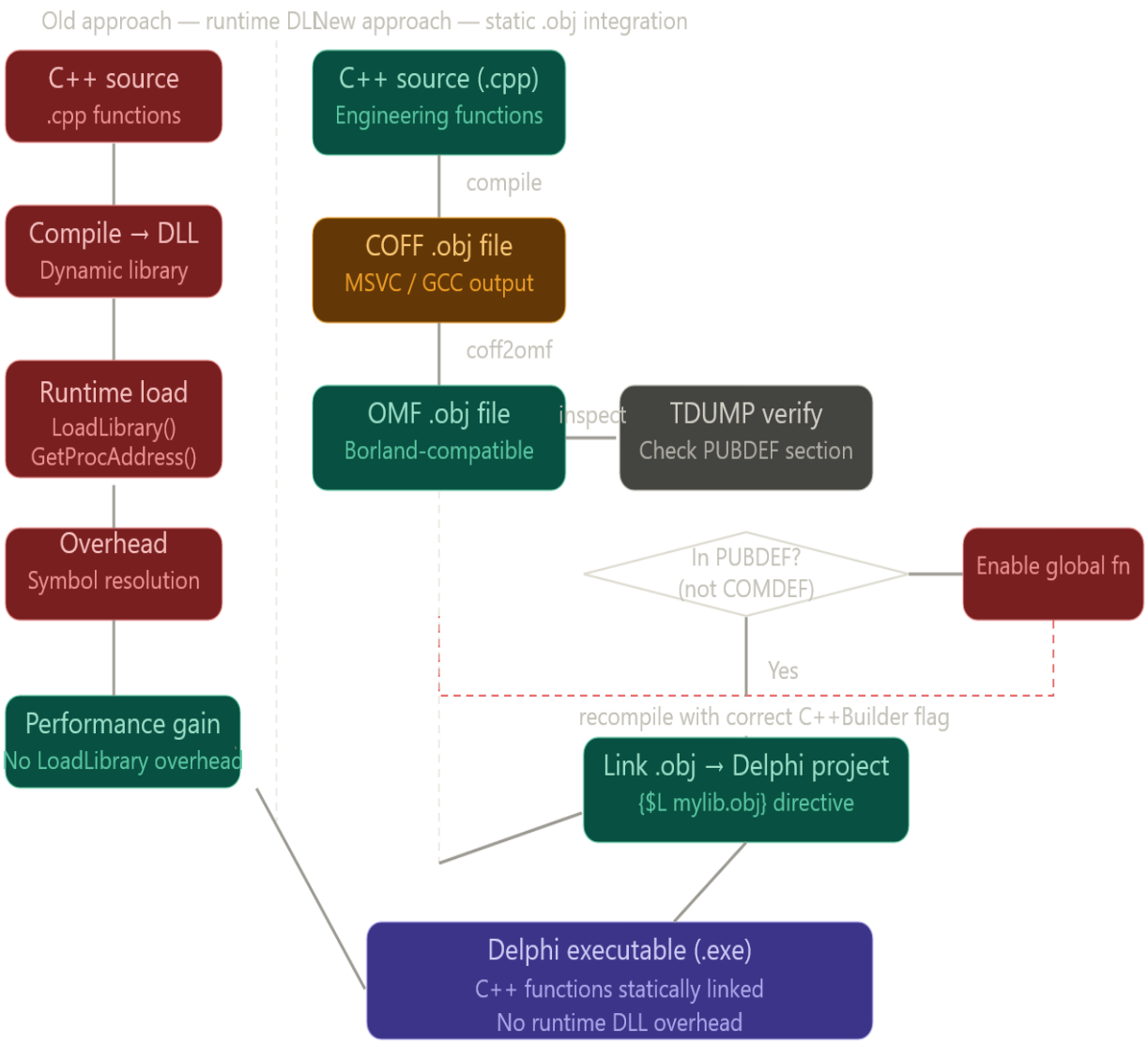


Figure 3. Approaches for Linking .obj Files

\

Complete System Architecture of Camera Intensity Balancer

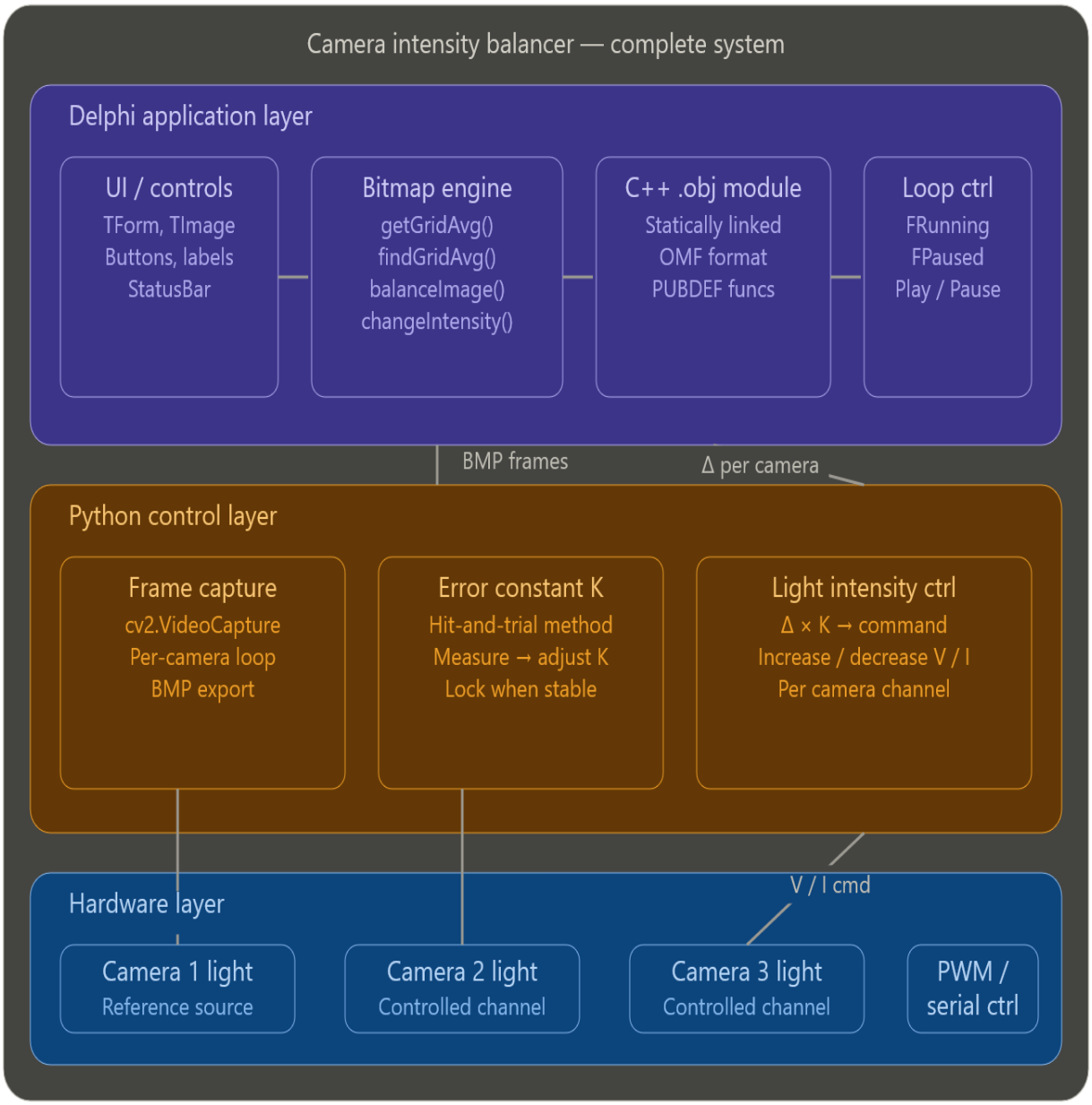


Figure 4. Complete Architecture diagram

Prototype Image

Figure 5. Imaginary Prototype

Chapter 4 : Research Challenge: Integration of C++ Functions with Delphi

During the development of the project, the company provided several pre-built functions written in the C++ programming language. These functions are part of the organization's internal tools and are used in various engineering calculations and project-related operations. Traditionally, these functions were accessed through compiled DLL (Dynamic Link Library) or executable (.exe) files. The Delphi-based system would call these DLLs or executables whenever the functions were required.

However, this approach introduced additional overhead because calling functions from external DLLs or executables requires dynamic loading, symbol resolution, and memory management during runtime. This process increases execution time compared to directly linking compiled code with the main application. Therefore, the company proposed a research task to investigate whether these C++ functions could be compiled into object files (.obj) and directly integrated into the Delphi project. Using object files could potentially improve performance by eliminating runtime linking overhead.

The primary objective of this research was to determine whether Delphi could directly utilize compiled C++ object files and, if possible, identify the necessary steps to achieve compatibility.

Conversion of C++ Code to Object Files

The first step in the research process involved compiling the C++ source code provided by the project guide using a C++ compiler. A compiler is a program that translates high-level programming language code into low-level machine instructions that can be executed by the computer. During compilation, the compiler generates an intermediate file called an object file (.obj), which contains machine-level instructions but is not yet a complete executable program.

Initially, when the C++ code was compiled, the generated object file was in the COFF (Common Object File Format). COFF is a widely used object file format supported by modern compilers such as Microsoft Visual C++ and GCC. It organizes compiled code into structured sections containing machine instructions, symbol tables, and debugging information.

However, Delphi and older Borland development tools are designed to work with a different object file format known as OMF (Object Module Format). Because of this difference, the COFF object file generated by the compiler could not be directly linked with the Delphi project.

To resolve this compatibility issue, the COFF object file was converted into OMF format. The OMF format is compatible with the Borland and Delphi linking mechanisms and allows the Delphi compiler to recognize external functions defined in object files.

DLL vs Object File Performance

One of the major motivations behind this research was performance optimization. When functions are stored in DLL files, the application must load the DLL during runtime and locate the function addresses dynamically before executing them. This process is known as dynamic linking and introduces additional processing overhead.

In contrast, when functions are compiled into object files (.obj) and linked during the

application build process, the code becomes part of the final executable. This method is known as static linking. Since the functions are already integrated into the application during compilation, the system does not need to perform runtime loading or symbol resolution. As a result, execution becomes faster and more efficient compared to DLL-based calls.

For systems that frequently call computational functions, using object files can significantly reduce execution time and improve overall system performance.

Symbol Definition Issue in Object Files

After successfully converting the object file into the required format and linking it with the Delphi project, another technical challenge was encountered. To verify the contents of the object file, the Turbo Dump (TDUMP) tool was used with the following command:

tdump -oiPUBDEF

This command displays the PUBDEF (Public Definitions) section of the object file. Public definitions represent functions that are globally accessible and can be linked from external programs such as Delphi.

However, during inspection it was discovered that the required C++ functions were not listed under the PUBDEF section. Instead, they were appearing under COMDEF (Common Definitions).

The COMDEF section typically contains definitions for uninitialized global variables rather than callable functions. Since the required functions were not present in the PUBDEF section, Delphi could not recognize them for linking purposes. This created a significant obstacle in integrating the compiled object files with the Delphi project.

Compiler Configuration and Solution

To resolve this issue, further investigation was conducted into the C++ Builder compiler settings. After analyzing the compiler configuration options, a specific setting related to global function segmentation was identified.

This compiler option instructs the compiler to place global functions into segments that are exported as public definitions. By enabling this setting and recompiling the C++ source code, the functions were correctly placed into segments that could be recognized by external linkers.

After recompilation, the object file was analyzed again using Turbo Dump (TDUMP). This time, the required functions appeared under the PUBDEF section, confirming that they were now publicly accessible.

Final Integration with Delphi

Once the object file contained the correct public definitions, it was successfully linked with the Delphi project. The Delphi application was then able to directly call the compiled C++ functions without relying on DLL files.

Additionally, further research was conducted to ensure compatibility between C++ data types and Delphi data structures. Since object files contain raw machine-level code, it was important to ensure that parameters passed between the two languages followed compatible memory layouts and calling conventions.

Through careful analysis and testing, the appropriate mappings between C++ data types and Delphi data types were established. This ensured that function parameters, return values, and data structures could be exchanged correctly between the two environments.

Research Outcome

At the end of the research, it was successfully demonstrated that C++ functions can be compiled into object files, converted into an OMF-compatible format, and directly integrated into a Delphi application. This approach eliminates the need for DLL-based function calls and significantly improves execution performance.

The research also provided insights into compiler behavior, object file formats, and cross-language compatibility, which were essential in overcoming the technical challenges encountered during the integration process.

Chapter 5 : Implementation, Testing, and Maintenance

5.1 Introduction to Languages, IDEs, Tools, Technologies Used for Implementation

The **Image Intensity Balancer System** is implemented as a desktop-based application designed to assist machine vision systems used in industrial environments. The system processes images captured from multiple cameras and balances their brightness levels using an automated algorithm.
The following programming languages, tools, and technologies were used during the development and implementation of the system.

Programming Languages

- **Delphi (Object Pascal)**
  Delphi is used as the primary programming language to develop the desktop application. It is responsible for implementing the graphical user interface, image processing algorithms, and system control logic. The application uses VCL components such as TForm, TImage, TPanel, and TButton to provide an interactive interface for loading images and controlling the intensity balancing process.
- **C++**
  Certain internal computational functions provided by the organization are written in C++. Research was conducted to compile these functions into object files and integrate them directly into the Delphi application to improve execution performance.
- **Python**
  Python was used during additional experimental work at STPL to control hardware lighting intensity and perform video frame extraction from camera feeds for image processing experiments.

IDE and Development Environment

- **RAD Studio (Delphi IDE)**
  RAD Studio is used as the main development environment for building the desktop application. It provides tools for GUI design, code editing, debugging, and compilation.
- **VCL (Visual Component Library)**
  The VCL framework is used to build the graphical user interface of the application. It provides visual components that simplify the creation of Windows desktop applications.

Image Processing Techniques

The application performs several image processing operations including:

- Grid-based image segmentation
- Pixel intensity averaging
- Brightness comparison between images
- Iterative intensity correction

These operations are implemented directly in Delphi using **bitmap manipulation and pointer-based pixel access**.

Compiler and Binary Tools

- **C++ Compiler**
  Used to compile C++ source code into object files.
- **Turbo Dump (TDUMP)**
  Used to inspect object files and verify symbol definitions such as PUBDEF sections.
- **COFF to OMF Conversion Tools**
  Used to convert object files from COFF format to OMF format so they can be linked with Delphi applications.

Design and Documentation Tools

The following tools were used to design diagrams and documentation for the project:

- **Draw.io / Lucidchart** – for UML diagrams, system architecture diagrams, and flowcharts
- **Microsoft Word** – for preparing project documentation and reports

Testing and Experimental Tools

- Sample camera images were used to test brightness balancing.
- Experimental scripts written in **Python** were used to analyze brightness behavior and adjust hardware lighting parameters.

5.2 Coding Standards of Language Used

To maintain code quality, readability, and maintainability, several coding standards were followed during the development of the Intensity Balancer System.

Naming Conventions

- Variables and functions follow **camelCase or PascalCase** naming conventions depending on their usage.
- Class names and component names use **PascalCase** for clarity.
- Constants are defined in **UPPER_CASE** format to distinguish them from variables.

Example from the implementation:
findGridAvg
changeIntensity
balanceImage

Code Documentation

- Inline comments are added throughout the source code to explain important operations and algorithms.
- Each function contains descriptive naming that indicates its purpose.

For example:

- getGridAvg() – calculates the average brightness of a grid region.
- balanceImage() – adjusts pixel intensity based on computed error values.
- changeIntensity() – modifies RGB pixel values to balance brightness.

Code Formatting

Consistent formatting rules were followed during implementation:

- Proper indentation is used to improve code readability.
- Related code blocks are grouped together logically.
- Functions performing similar operations are organized within the same unit.

Performance Optimization

Several techniques were used to improve processing performance:

- **Pointer-based pixel access** using ScanLine and PRGBTriple.
- Direct manipulation of RGB values instead of high-level image operations.
- Grid-based processing to reduce computational complexity.

Example concept used in the implementation:

- Accessing pixel data using pointers instead of calling slow pixel methods.

This approach significantly improves processing speed when working with large images.

Modularity

The application is structured in a modular way where different functionalities are separated into specific procedures.
Examples include:

- findGridAvg() – grid brightness comparison
- balanceImage() – brightness correction
- showAvg() – display brightness values
- changeIntensity() – pixel-level intensity modification

This modular structure makes the code easier to understand and maintain.

Error Handling

Basic error handling mechanisms are implemented to ensure system stability.

- try-except blocks are used when loading images to handle invalid files or loading errors.
- Input validation is performed to ensure correct grid sizes and image dimensions.

Example validations include:

- Checking if exactly **three images** are selected.
- Ensuring all images have **identical dimensions**.
- Validating that grid size is greater than zero.

User Interface Feedback

The system provides real-time status updates to users during processing.

Examples include:

- Displaying the current **iteration number**.
- Showing **brightness difference values** between cameras.
- Updating system status messages during processing, pause, and completion.

Maintainability

The system is designed so that future improvements can easily be integrated.

Possible extensions include:

- Integration with real-time camera feeds
- Hardware lighting control
- Automated machine vision pipelines

The modular design and clear coding standards ensure that the system can be maintained and expanded in future development stages.

Chapter 6. Results and Discussions

6.1 User Interface Representation – Intensity Balancer System

The **Intensity Balancer System** provides a graphical desktop interface developed using **Delphi (Object Pascal) with the VCL framework**. The user interface is designed to allow operators to load multiple camera images, analyze brightness values, and automatically balance intensity differences between cameras. The interface focuses on simplicity and clarity so that engineers or machine operators can easily operate the system without deep knowledge of image processing.

The main interface consists of three image display panels representing the **Reference Camera**, **Camera 2**, and **Camera 3**. The reference camera image acts as the baseline for brightness comparison, while the other two images are adjusted during the balancing process. The system also provides controls such as **Load Images**, **Process**, **Play**, and **Pause**, allowing the user to control execution of the algorithm.

Another important feature of the interface is the **grid visualization of intensity values**. During processing, the image is divided into smaller grid cells, and the average brightness value of each region is displayed on the screen. This allows users to visually observe the balancing process and verify how intensity differences are corrected across images.

The interface also includes a **status bar** that displays important information such as iteration count, brightness difference values, and completion messages. This feedback helps the user monitor system performance and understand the progress of the balancing algorithm.

Overall, the user interface provides a clear and interactive environment for testing and demonstrating the effectiveness of the **grid-based intensity balancing algorithm** implemented in the project.

6.1.1 Brief Description of Various Components of the System

The Intensity Balancer System consists of several important functional components that work together to perform image brightness equalization.

Image Loading Module

This module allows users to select and load **three images simultaneously** using a file dialog. The images represent different camera inputs captured from the same scene. The system verifies that all images have the same dimensions before processing begins.

Reference Image Module

The first image acts as the **reference camera image**. The brightness values of this image are used as the baseline against which other camera images are compared.

Grid-Based Analysis Module

In this module, each image is divided into smaller square regions based on the **cell size specified by the user**. The average brightness value for each region is calculated to determine local intensity differences between images.

Intensity Balancing Algorithm

The balancing module calculates the difference in brightness between corresponding grid regions of the reference image and other camera images. Using an **iterative half-step correction method**, pixel intensities are gradually adjusted until the difference becomes minimal.

Visualization Module

This module overlays numerical brightness values on the images so that users can observe the average intensity of each grid cell. This visual representation helps verify how the balancing process is improving brightness consistency.

Execution Control Module

The system provides **Play and Pause controls** to manage the balancing process. This allows the user to pause the algorithm during execution and resume it when required.

Status Monitoring Module

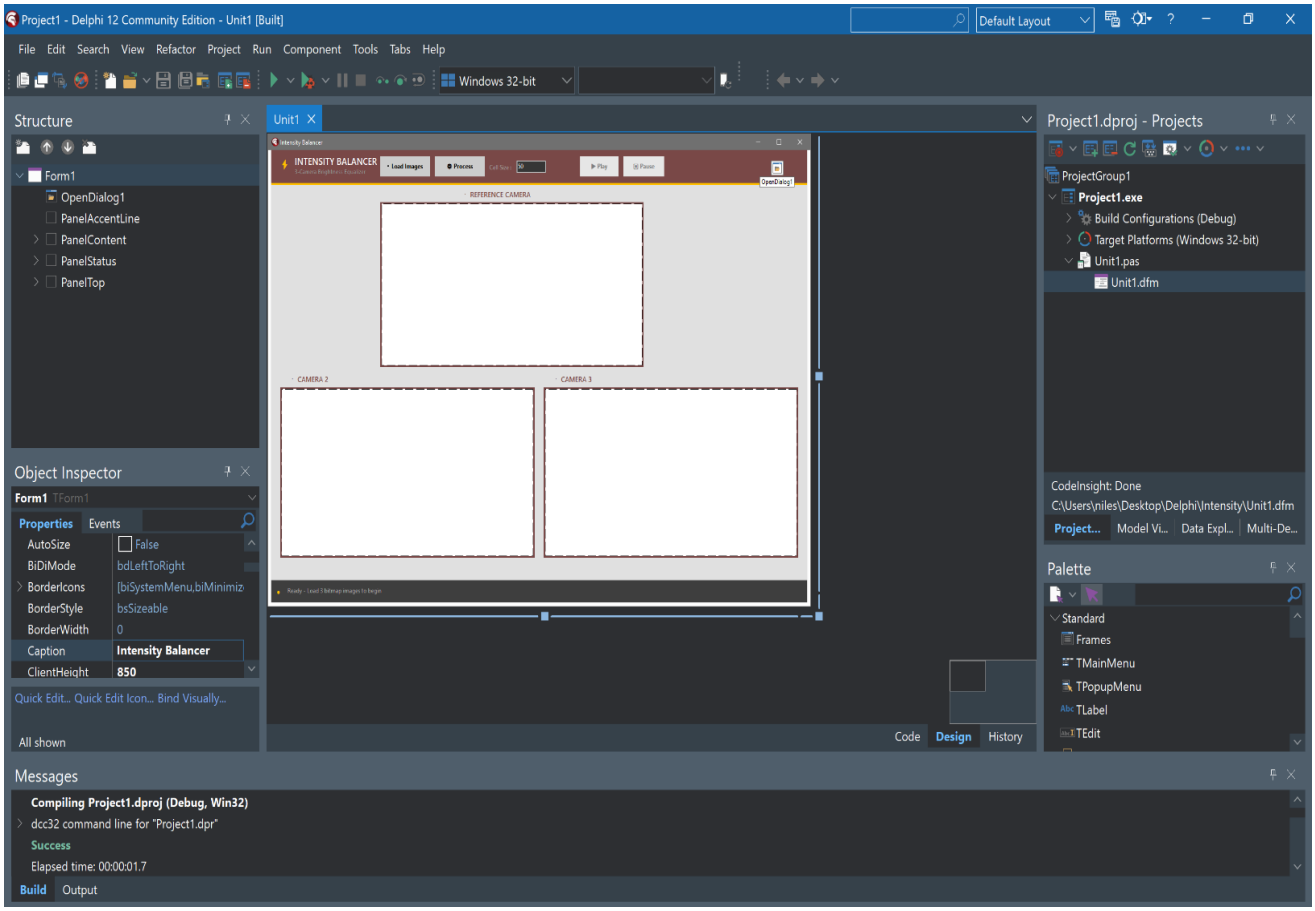
The status bar at the bottom of the interface displays information such as iteration number, intensity difference values, and completion status of the balancing process.

6.1 Snapshots

1. Application Interface in Delphi IDE

This screenshot shows the **development environment of the Intensity Balancer System inside Delphi IDE**. The form designer is used to arrange graphical components such as panels, labels, buttons, and image containers.

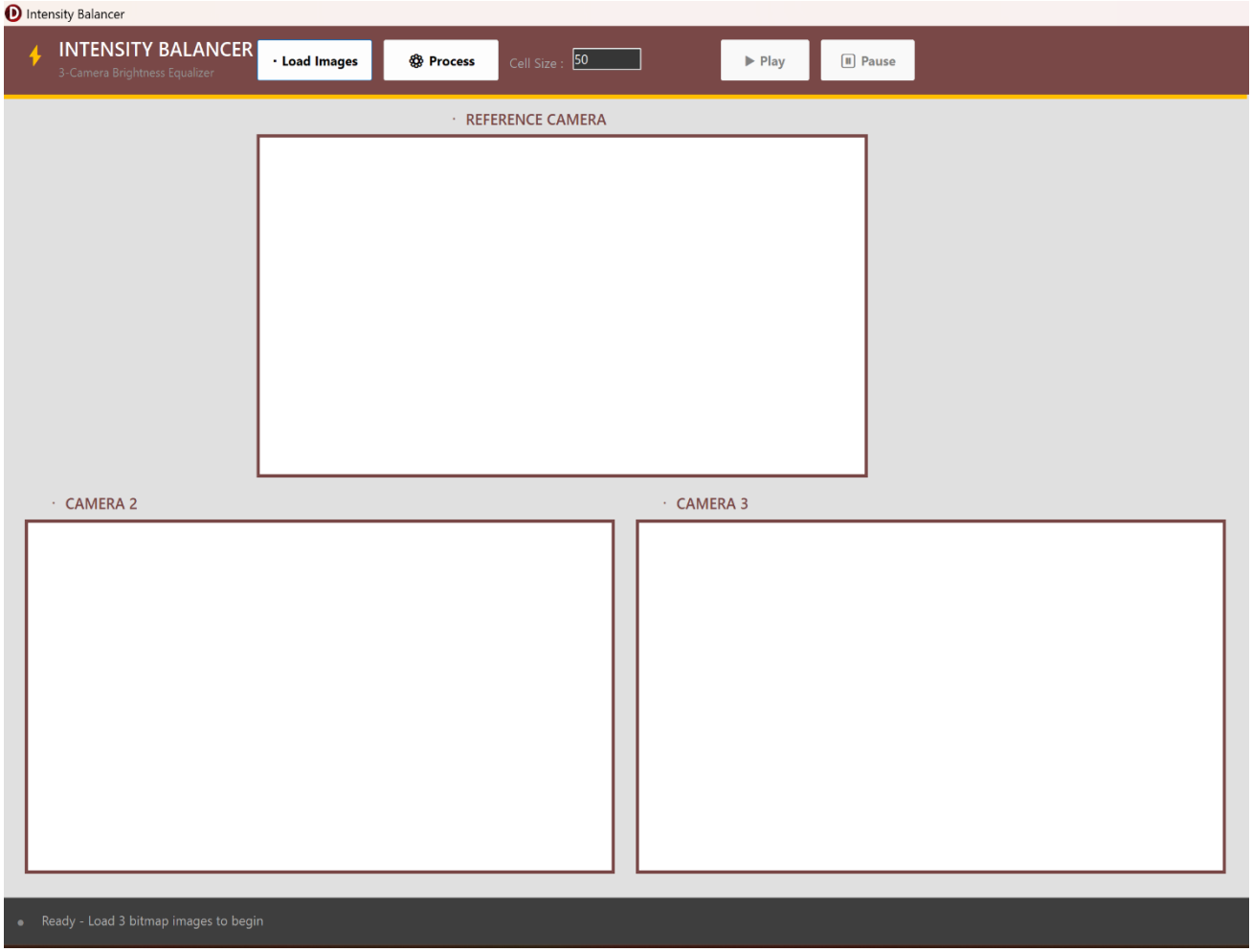
The interface layout contains three major display areas representing the three camera inputs. Control buttons such as **Load Images**, **Process**, **Play**, and **Pause** are placed in the header section to provide easy access to system operations. The Delphi environment allows rapid visual development and simplifies GUI design for desktop applications.



2. Initial Application Window

This screenshot shows the application immediately after execution. At this stage, the system is ready to accept image inputs but no images have been loaded yet. The three empty panels indicate the placeholders for the reference camera and the two additional camera images.

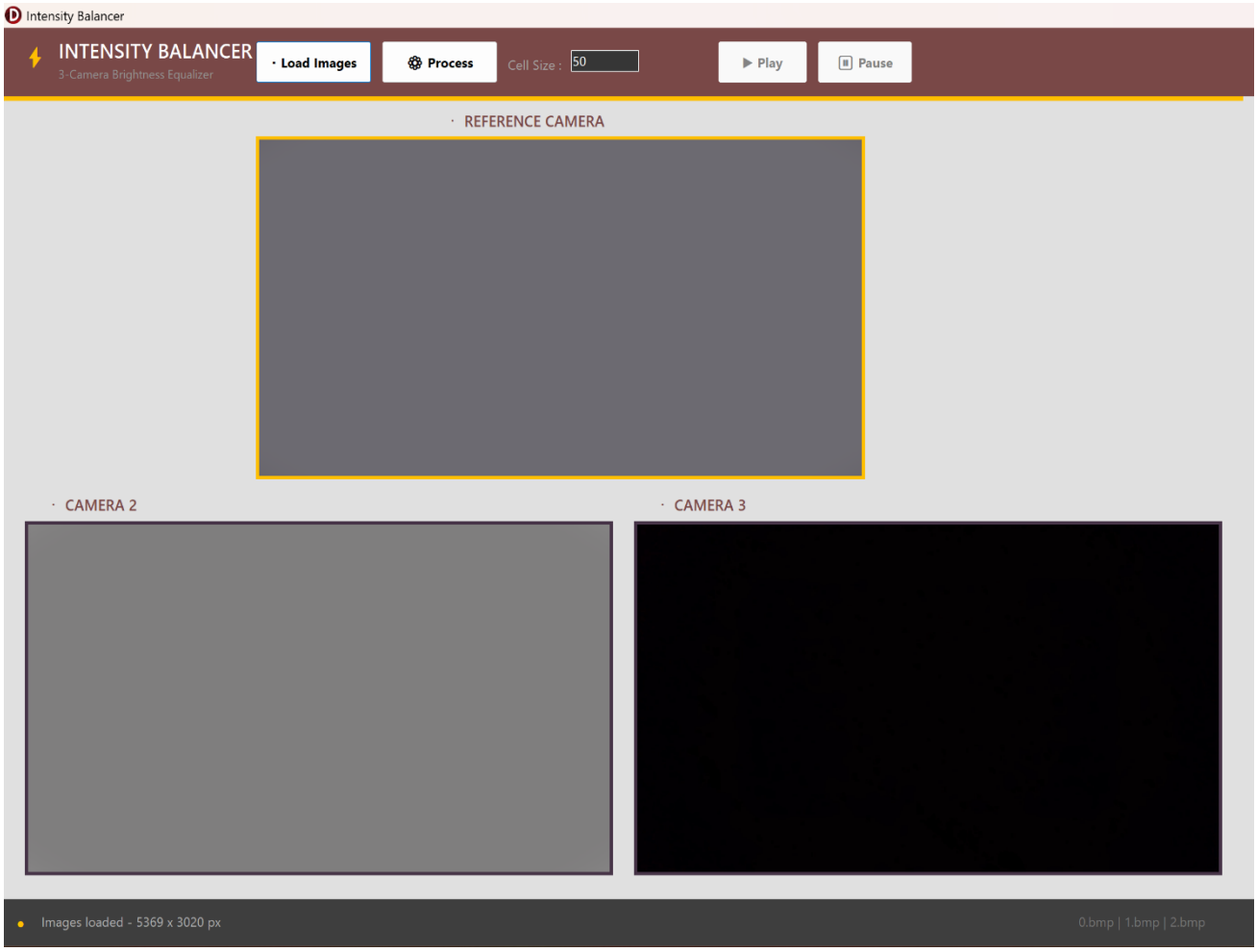
The status bar at the bottom displays a message instructing the user to load three images before starting the processing stage. This ensures that the system receives the required inputs before executing the balancing algorithm.



3. Images Loaded into the System

After selecting three images, they are loaded into the application and displayed in their respective panels. The **reference camera image** is displayed at the top, while the other two camera images are shown in the lower panels.

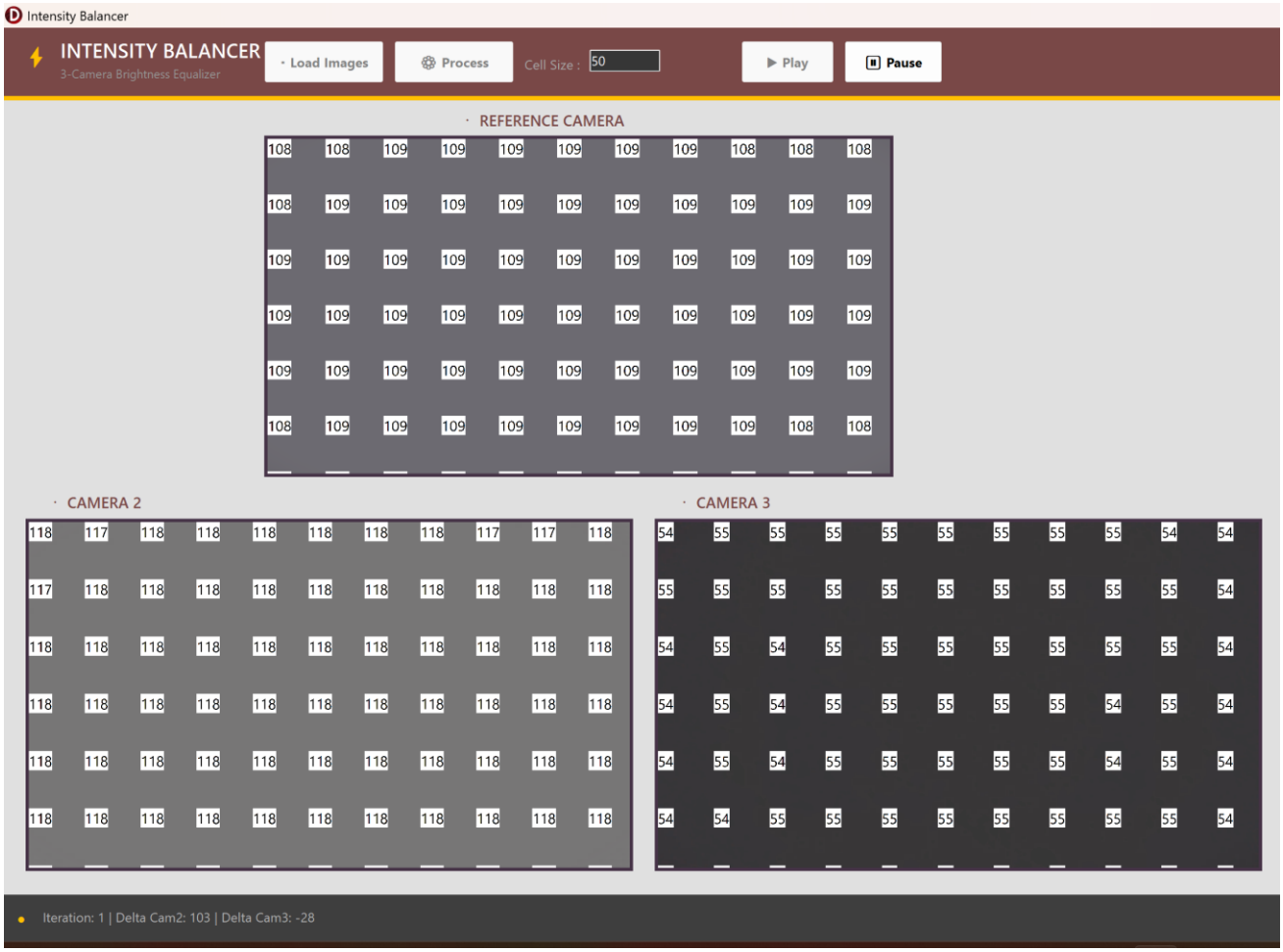
The system verifies that all images have identical dimensions before allowing further processing. Once the images are successfully loaded, the user can specify the **grid cell size** and initiate the balancing process.



4. Grid-Based Intensity Analysis

In this stage, the system divides each image into smaller grid cells and calculates the **average brightness value of each cell**. These values are displayed directly on the images.

This visualization helps identify intensity differences between the reference image and other camera images. The grid-based approach enables localized brightness correction rather than adjusting the entire image uniformly.

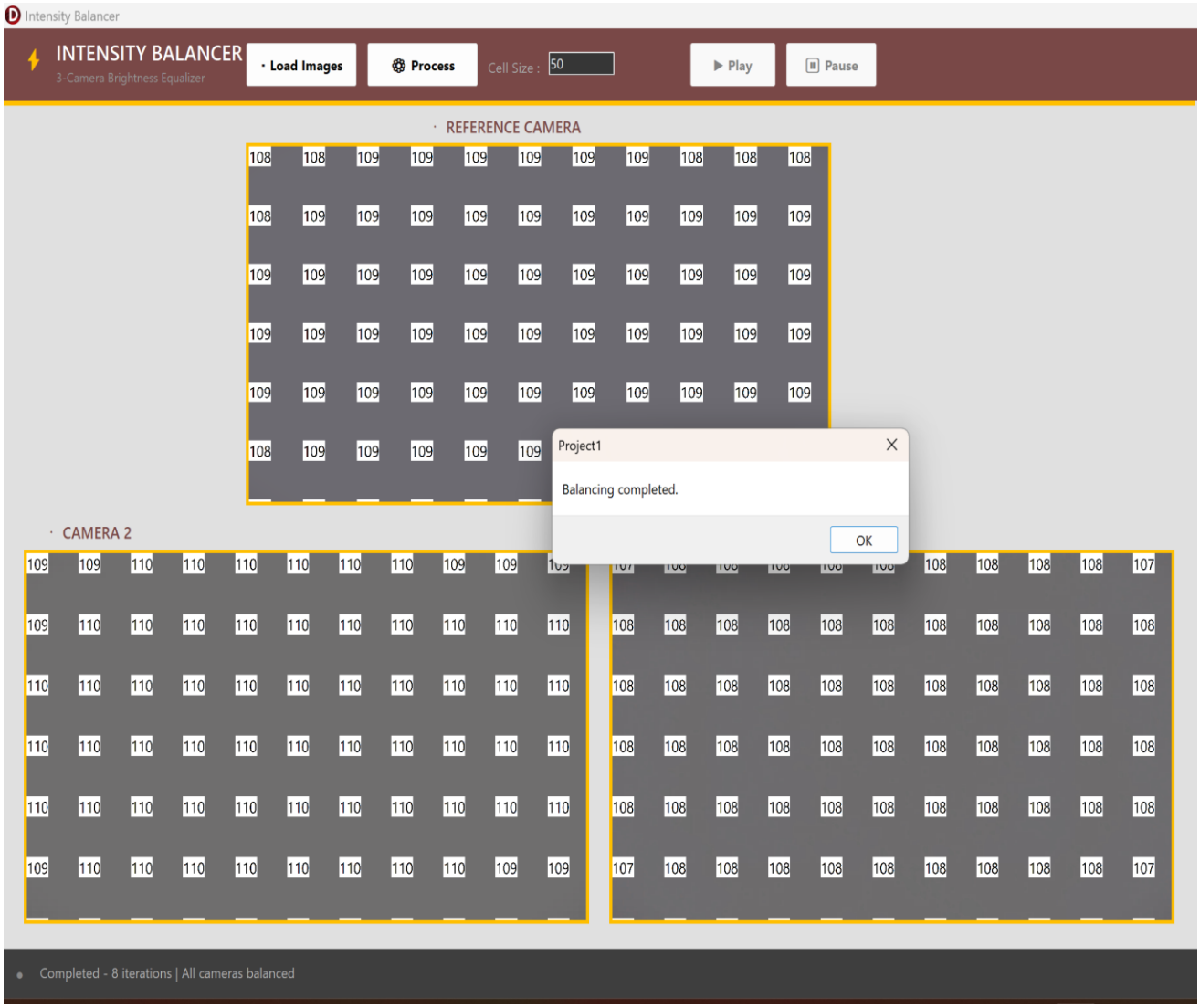


5. Iterative Intensity Balancing and Final Output

During the balancing process, the algorithm iteratively adjusts the brightness values of the target images. The system first calculates the intensity difference between the **reference image** and the other camera images for each grid region. Based on this difference, a **half-step correction** is applied to gradually reduce the brightness mismatch.

With every iteration, the intensity values of the target images move closer to the reference image values. The **status bar displays iteration details and brightness difference values**, allowing the user to monitor the progress of the balancing process in real time.

Once the intensity differences between corresponding grid regions fall below the predefined threshold, the system automatically stops the balancing algorithm. At this stage, the final output images show **balanced brightness levels across all cameras**, demonstrating the effectiveness of the implemented intensity balancing method.



Chapter 7. Workflow during Internship

• The internship duration was **02-01-2026 to 31-06-2026 (six months)**.

• During the **first and second week (02/01 – 11/01)**, the onboarding process was completed. Since I was working under the **Research and Development (R&D) Department**, I was assigned a workstation specifically configured for development and experimentation. One system was kept **offline for secure development**, while another system with internet access was provided for documentation and research purposes.

• As the **R&D department handles sensitive research work**, strict security policies were followed. Employees are not allowed to take **source code, research documents, or white papers outside the office environment** to maintain confidentiality of internal technologies.

• During the remaining weeks of **January (13/01 – 31/01)**, I started learning the **Delphi programming language and RAD Studio development environment**. I practiced basic programming concepts, file operations, graphical interface design, and event handling. To strengthen my understanding, I developed small experimental applications such as **simple image viewers and GUI-based utilities**.

• In the **first three weeks of February (01/02 – 22/02)**, I explored more advanced features of Delphi including **bitmap handling, pixel manipulation, pointer-based memory access, and graphical components used for image display**. During this phase, I also studied the basics of **digital image processing**, particularly brightness calculation and pixel intensity operations.

• From **24/02 onward**, I was assigned the main research task of developing an **Image Intensity Balancer System** for machine-vision cameras used in industrial diamond processing machines. The objective was to design a software system capable of analyzing brightness differences between images captured by multiple cameras and automatically balancing their intensity levels.

• Between **24/02 – 01/03**, I designed the **basic architecture of the application** and implemented the graphical interface where users can load multiple images and visualize brightness information. I also designed the **grid-based analysis structure** used for localized brightness comparison.

• During the **first week of March (02/03 – 08/03)**, I started implementing the core **image processing algorithms**. This included dividing images into grid regions, calculating average brightness values using the **weighted RGB luminance formula**, and detecting intensity differences between reference and target images.

• In the **second and third week of March (10/03 – 22/03)**, I implemented the **iterative half-step correction algorithm** that gradually adjusts brightness values to minimize the difference between images. Performance optimization was performed using **pointer-based pixel access through ScanLine and PRGBTriple structures**.

• During the **last week of March and early April (24/03 – 05/04)**, a research challenge was introduced regarding the use of **existing C++ functions provided by the company**. These functions were originally accessed through **DLL or executable modules**, which caused additional runtime overhead.

• To improve performance, I conducted research to determine whether **C++ code could be compiled into object (.OBJ) files and directly linked with the Delphi application**. This involved compiling the C++ code using a C++ compiler, analyzing the generated object files, and studying differences between **COFF (Common Object File Format)** and **OMF (Object Module Format)**.

• After compiling the C++ code, the generated **COFF object files were converted to OMF format**, which is compatible with the Delphi compiler. Using the **Turbo Dump (TDUMP) tool**, I analyzed the object file structure to verify whether functions were available under **PUBDEF (Public Definitions)**.

• Initially, many functions appeared under **COMDEF instead of PUBDEF**, making them inaccessible for linking with Delphi. After further research, a **C++ Builder compiler setting called "Global Functions in Segment"** was identified and enabled, which allowed the functions to be exported correctly.

• After resolving these challenges, the **OBJ files were successfully integrated with the Delphi application**, enabling faster execution compared to DLL calls. Additional testing was performed to ensure compatibility between **C++ data types and Delphi data structures**, since object files contain raw compiled code.

• Currently, the system is being tested and improved by performing **algorithm optimization, stability testing, and validation of brightness balancing results** across different image inputs.

Chapter 8. Conclusion and Future Scope

8.1 Conclusion

The **Image Intensity Balancer System** developed as part of this internship project at **STPL** addresses the challenge of brig htness inconsistencies between images captured by multiple cameras in industrial machine-vision environments. Variations in lighting conditions and camera sensor characteristics often result in uneven intensity levels, which can affect the accuracy and reliability of image-based analysis systems.

The developed system provides a structured approach for analyzing and correcting brightness differences between images. By dividing images into smaller grid regions and calculating perceptual luminance using the **weighted RGB formula**, the system identifies localized intensity differences and applies an **iterative half-step correction algorithm** to gradually balance brightness levels. This approach improves visual consistency across camera outputs and enhances the quality of image data used for further processing.

Additionally, the project involved research on integrating **C++ computational functions with Delphi applications** through **object (.OBJ) files**. By compiling C++ code and converting it into a compatible object format, these functions were successfully linked directly with the Delphi application, reducing the overhead associated with DLL-based calls. This research contributed to improved performance and demonstrated an efficient method of integrating low-level compiled code within a high-level application environment.

Overall, the developed system demonstrates that automated intensity balancing can improve the reliability of machine-vision systems used in industrial applications and reduce dependence on manual adjustments.

8.2 Future Scope

Although the current system successfully demonstrates the concept of automated image intensity balancing, several enhancements can be implemented in the future to further improve its capabilities.

One possible improvement is the **integration of the algorithm directly into industrial machine-vision systems**, enabling real-time brightness balancing during camera operation. This would allow images to be corrected automatically as they are captured, improving efficiency and reducing the need for manual intervention.

The system can also be extended to support **real-time video stream processing**, allowing continuous intensity balancing for live camera feeds instead of static images. This would make the system more suitable for automated inspection and monitoring environments.

Another potential enhancement is the incorporation of **advanced image processing techniques or machine learning algorithms** to dynamically adapt to varying lighting conditions and improve balancing accuracy. Additionally, implementing **parallel processing or GPU-based computation** could significantly improve processing speed when handling high-resolution images.

With these improvements, the Image Intensity Balancer System has the potential to evolve into a more advanced solution for **industrial imaging, machine vision, and automated inspection systems**, supporting the growing technological needs of modern manufacturing environments.

Data Dictionary

Term / Data Element	Description
Camera Feed	Image data captured from each camera in the multi-camera setup
Reference Image	Base image used to compare brightness levels with other images
Target Image	Image whose brightness is adjusted to match the reference image
Pixel	Smallest unit of a digital image containing RGB color values
RGB Values	Red, Green, and Blue intensity components of a pixel
Luminance	Perceived brightness of a pixel calculated using weighted RGB values
Grid Segmentation	Process of dividing an image into smaller cells for localized analysis
Grid Cell	Individual segment of the image used for brightness computation
Average Intensity	Mean brightness value of pixels within a grid cell
Intensity Error	Difference between brightness values of reference and target cells
Threshold	Maximum allowable brightness difference before correction stops
Iterative Adjustment	Repeated correction process applied until brightness converges
Brightness Correction	Adjustment applied to pixel values to balance intensity
Image Buffer	Memory area storing pixel data during processing
ScanLine Pointer	Direct memory access method used for faster pixel processing
Balanced Output	Final image with normalized brightness levels across cameras

References

- https://www.marcocantu.com/objectpascalhandbook/
- https://www.packtpub.com/en-pl/product/delphi-gui-programming-with-firemonkey-9781788624176
- https://docwiki.embarcadero.com/RADStudio/Athens/en/Delphi_Language_Reference
- https://docwiki.embarcadero.com/RADStudio/en/Helpful_Resources_for_New_Users
- https://learndelphi.org/delphi-language-guide/
- https://en.wikibooks.org/wiki/Delphi_Programming
- https://www.packtpub.com/product/mastering-delphi-programming-a-complete-reference-guide/9781838989118
- https://delphi-books.com/en/Getting-started-with-RAD-Studio.html